

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

VICTOR HUGO DOS SANTOS CAVALCANTE

LISTA 02 - EXERCÍCIO SQL

CAMPOS DO JORDÃO

2025

VICTOR HUGO DOS SANTOS CAVALCANTE

LISTA 02 - EXERCÍCIO SQL

Projeto desenvolvido como requisito parcial para avaliação na disciplina de Banco de Dados 2, correspondente à segunda lista sobre a execução dos exercício do livro base da disciplina (Sistemas de Banco de Dados: Projeto, Implementação e Administração, dos autores Peter Rob e Carlos Coronel).

Orientador: Paulo Giovani de Faria Zeferino

CAMPOS DO JORDÃO

2025

RESUMO

Este relatório apresenta o desenvolvimento prático da segunda lista para a disciplina de Banco de Dados 2, do curso de Análise e Desenvolvimento de Sistemas. O objetivo principal do trabalho é aplicar os conceitos teóricos de modelagem de dados e manipulação de Linguagem de Consulta Estruturada (SQL). A metodologia adotada envolve a criação dos modelos conceitual, lógico e físico baseados em um estudo de caso específico (Figura P7.15 da literatura de Peter Rob e Carlos Coronel), seguida pela inserção de registros para popular as tabelas. Posteriormente, são resolvidos e documentados os exercícios práticos de 16 a 33, abrangendo consultas complexas com o uso de junções (*JOINS*), agrupamentos (*GROUP BY*), subconsultas e funções de agregação. Os resultados obtidos demonstram a correta estruturação do banco de dados relacional e a eficácia das consultas formuladas para a extração de informações estratégicas, consolidando o aprendizado prático em implementação e administração de banco de dados.

Palavras-chave: Banco de Dados Relacional. Modelagem de Dados. SQL. Consultas.

ABSTRACT

This report presents the practical development of the second list of the semester for the Database 2 course, part of the Systems Analysis and Development program. The main objective of the work is to apply the theoretical concepts of data modeling and Structured Query Language (SQL) manipulation. The methodology involves the creation of conceptual, logical, and physical models based on a specific case study (Figure P7.15 from Peter Rob and Carlos Coronel's literature), followed by the insertion of records to populate the tables. Subsequently, practical exercises 16 to 33 are solved and documented, covering complex queries using joins, grouping, subqueries, and aggregation functions. The results obtained demonstrate the correct structuring of the relational database and the effectiveness of the queries formulated for extracting strategic information, consolidating practical learning in database administration and implementation.

Keywords: Relational Database. Data Modeling. SQL. Queries.

LISTA DE TABELAS

TABELA 1 – CUSTOMER	14
TABELA 2 – VENDOR	14
TABELA 3 – INVOICE	15
TABELA 4 – LINE	15
TABELA 5 – PRODUCT	16

LISTA DE SIGLAS

DML Data Manipulation Language

SGBD Sistema de Gerenciamento de Banco de Dados

SUMÁRIO

1	INTRODUÇÃO _____	6
2	MODELOS CONCEITUAL, LÓGICO E FÍSICO _____	7
2.1	Modelo Conceitual _____	7
2.2	Modelo Lógico _____	7
2.3	Modelo Físico _____	8
3	DADOS PARA A CRIAÇÃO DO BANCO _____	8
4	EXERCÍCIO 16 ATÉ 33 _____	10
5	CONCLUSÃO _____	18

1 INTRODUÇÃO

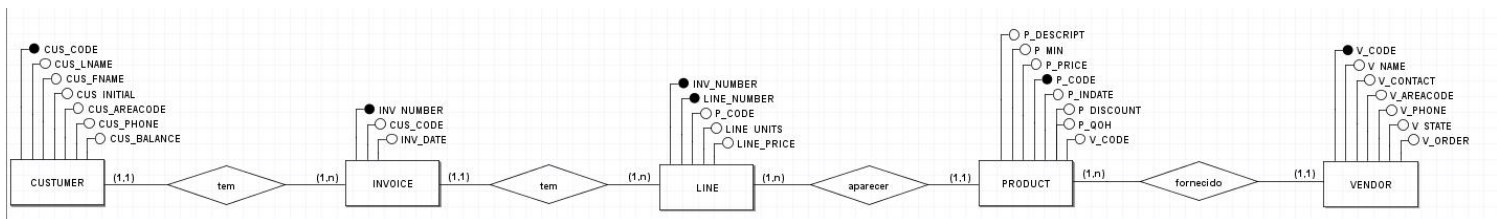
Este relatório apresenta o desenvolvimento prático da segunda lista da disciplina de Banco de Dados 2. O objetivo central do trabalho é aplicar os fundamentos teóricos de modelagem de dados por meio da elaboração dos diagramas conceitual, lógico e físico, utilizando como base o estudo de caso da Figura P7.15 da obra "*Sistemas de Banco de Dados*", dos autores Peter Rob e Carlos Coronel.

Além da estruturação arquitetural e da inserção dos registros iniciais, o documento detalha a resolução analítica dos exercícios práticos de 16 a 33. Para cada questão, são apresentados o enunciado proposto, o script SQL (*Data Manipulation Language*

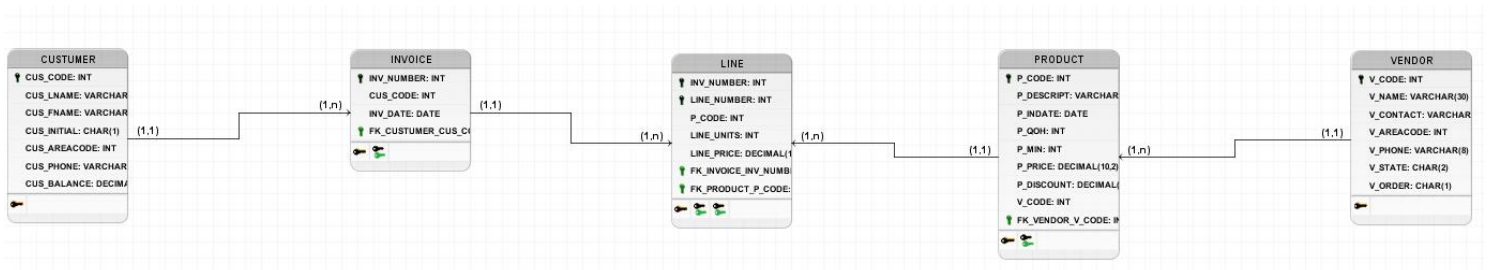
- DML) desenvolvido e a captura de tela com os resultados obtidos no SGBD. Dessa forma, o relatório busca demonstrar o domínio prático na criação de bases de dados relacionais e na execução de consultas complexas para a extração de informações.

2 MODELOS CONCEITUAL, LÓGICO E FÍSICO

2.1 Modelo Conceitual



2.2 Modelo Lógico



2.3 Modelo Físico

```

Exibição de modelo físico
1. /* Lógico_1: */
2.
3. CREATE TABLE CUSTOMER (
4.   CUS_CODE INT PRIMARY KEY,
5.   CUS_LNAME VARCHAR(30),
6.   CUS_FNAME VARCHAR(30),
7.   CUS_INITIAL CHAR(1),
8.   CUS_AREACODE INT,
9.   CUS_PHONE VARCHAR(8),
10.  CUS_BALANCE DECIMAL(10,2)
11. );
12.
13. CREATE TABLE INVOICE (
14.  INV_NUMBER INT PRIMARY KEY,
15.  CUS_CODE INT,
16.  INV_DATE DATE,
17.  FK_CUSTOMER_CUS_CODE INT
18. );
19.
20. CREATE TABLE LINE (
21.  INV_NUMBER INT,
22.  LINE_NUMBER INT,
23.  P_CODE INT,
24.  LINE_UNITS INT,
25.  LINE_PRICE DECIMAL(10,2),
26.  FK_INVOICE_INV_NUMBER INT,
27.  FK_PRODUCT_P_CODE INT,
28.  PRIMARY KEY (INV_NUMBER, LINE_NUMBER)
29. );
30.
31. CREATE TABLE PRODUCT (
32.  P_CODE INT PRIMARY KEY,
33.  P_DESCRIPTOR VARCHAR(80),
34.  P_INDATE DATE,
35.  P_QOH INT,
36.  P_MIN INT,
37.  P_PRICE DECIMAL(10,2),
38.  P_DISCOUNT DECIMAL(5,2),
39.  V_CODE INT,
40.  FK_VENDOR_V_CODE INT
41. );
42.

```

```

43. CREATE TABLE VENDOR (
44.  V_CODE INT PRIMARY KEY,
45.  V_NAME VARCHAR(30),
46.  V_CONTACT VARCHAR(30),
47.  V_AREACODE INT,
48.  V_PHONE VARCHAR(8),
49.  V_STATE CHAR(2),
50.  V_ORDER CHAR(1)
51. );
52.
53. ALTER TABLE INVOICE ADD CONSTRAINT FK_INVOICE_2
54.  FOREIGN KEY (FK_CUSTOMER_CUS_CODE)
55.  REFERENCES CUSTOMER (CUS_CODE)
56.  ON DELETE RESTRICT;
57.
58. ALTER TABLE LINE ADD CONSTRAINT FK_LINE_2
59.  FOREIGN KEY (FK_INVOICE_INV_NUMBER)
60.  REFERENCES INVOICE (INV_NUMBER)
61.  ON DELETE RESTRICT;
62.
63. ALTER TABLE LINE ADD CONSTRAINT FK_LINE_3
64.  FOREIGN KEY (FK_PRODUCT_P_CODE)
65.  REFERENCES PRODUCT (P_CODE)
66.  ON DELETE RESTRICT;
67.
68. ALTER TABLE PRODUCT ADD CONSTRAINT FK_PRODUCT_2
69.  FOREIGN KEY (FK_VENDOR_V_CODE)
70.  REFERENCES VENDOR (V_CODE)
71.  ON DELETE RESTRICT;

```

3 DADOS PARA A CRIAÇÃO DO BANCO

CREATE DATABASE bd_coronel;

```

CREATE TABLE CUSTOMER (
  CUS_CODE INT PRIMARY KEY,
  CUS_LNAME VARCHAR(30),
  CUS_FNAME VARCHAR(30),
  CUS_INITIAL CHAR(1),
  CUS_AREACODE INT,
  CUS_PHONE VARCHAR(8),
  CUS_BALANCE DECIMAL(10,2));
INSERT INTO CUSTOMER (CUS_CODE,
CUS_LNAME, CUS_FNAME, CUS_INITIAL, CUS_AREACODE, CUS_PHONE,
CUS_BALANCE) VALUES(10010, 'Ramos', 'Alfred', 'A', '615', '844-2573',
0.00),(10011, 'Dunne', 'Leona', 'K', '713', '894-1230', 0.00),(10012, 'Smith', 'Kathy', 'W',
'615', '894-2285', 345.86),(10013, 'Ortega', 'Paul', 'F', '615', '894-2180',
536.75),(10014, 'Orlando', 'Myron', NULL, '615', '222-1672', 0.00),(10015, 'O'Brien',
'Amy', 'B', '713', '442-3381', 0.00),(10016, 'Brown', 'James', 'G', '615', '297-1220',
221.19),(10017, 'Williams', 'George', NULL, '615', '290-2586', 768.93),(10018, 'Farris',
'Anne', 'O', '713', '382-7185', 216.55),(10019, 'Smith', 'Olette', 'K', '615', '297-3809',
0.00);
CREATE TABLE INVOICE (
  INV_NUMBER INT PRIMARY KEY,
  CUS_CODE INT,

```

```

    INV_DATE DATE);INSERT INTO INVOICE (INV_NUMBER, CUS_CODE,
INV_DATE) VALUES(1001, 10014, '2008-03-16'),(1002, 10011, '2008-03-16'),(1003,
10012, '2008-03-16'),(1004, 10011, '2008-03-17'),(1005, 10018, '2008-03-17'),(1006,
10014, '2008-03-17'),(1007, 10015, '2008-03-17'),(1008, 10011, '2008-03-
17');SELECT * FROM INVOICE;

```

```

CREATE TABLE LINE (
    INV_NUMBER INT,
    LINE_NUMBER INT,
    P_CODE VARCHAR(20),
    LINE_UNITS INT,
    LINE_PRICE DECIMAL(10,2),
    PRIMARY KEY (INV_NUMBER, LINE_NUMBER),
    FOREIGN KEY (INV_NUMBER) REFERENCES
INVOICE(INV_NUMBER));INSERT INTO LINE (INV_NUMBER, LINE_NUMBER,
P_CODE, LINE_UNITS, LINE_PRICE) VALUES(1001, 1, '13-Q2/P2', 1, 14.99),(1001,
2, '23109-HB', 1, 9.95),
(1002, 1, '54778-2T', 2, 4.99),
(1003, 1, '2238/QPD', 1, 38.95),(1003, 2, '1546-QQ2', 1, 39.95),(1003, 3, '13-Q2/P2',
5, 14.99),
(1004, 1, '54778-2T', 3, 4.99),(1004, 2, '23109-HB', 2, 9.95),
(1005, 1, 'PVC23DRT', 12, 5.87),
(1006, 1, 'SM-18277', 3, 6.99),(1006, 2, '2238/QPD', 1, 109.92),(1006, 3, '23109-HB',
1, 9.95),(1006, 4, '89-WRE-Q', 1, 256.99),
(1007, 1, '13-Q2/P2', 2, 14.99),(1007, 2, '54778-2T', 1, 4.99),
(1008, 1, 'PVC23DRT', 5, 5.87),(1008, 2, 'WR3/TT3', 3, 119.95),(1008, 3, '23109-HB',
1, 9.95);

```

```

CREATE TABLE PRODUCT (
    P_CODE VARCHAR(20) PRIMARY KEY,
    P_DESCRIPT VARCHAR(100),
    P_INDATE DATE,
    P_QOH INT,
    P_MIN INT,
    P_PRICE DECIMAL(10,2),
    P_DISCOUNT DECIMAL(5,2),
    V_CODE INT,
    FOREIGN KEY (V_CODE) REFERENCES VENDOR(V_CODE));INSERT INTO
PRODUCT (P_CODE, P_DESCRIPT, P_INDATE, P_QOH, P_MIN, P_PRICE,
P_DISCOUNT, V_CODE) VALUES('11QER/31', 'Power painter, 15 psi, 3-nozzle',
'2007-11-03', 8, 5, 109.99, 0.00, 25595),
('13-Q2/P2', '7.25-in. pwr. saw blade', '2007-12-13', 32, 15, 14.99, 0.05, 21344),
('14-Q1/L3', '8.00-in. pwr. saw blade', '2007-11-13', 18, 12, 17.49, 0.00, 21344),
('1546-QQ2', 'Hrd. cloth, 1/4-in., 2x50', '2008-01-15', 15, 8, 39.95, 0.00, 23119),
('1558-QW1', 'Hrd. cloth, 1/8-in., 3x50', '2008-01-15', 23, 5, 43.99, 0.00, 23119),
('2232/QTY', 'B&D jigsaw, 12-in. blade', '2007-12-30', 8, 5, 109.92, 0.05, 24288),
('2232/QWE', 'B&D jigsaw, 8-in. blade', '2007-12-24', 6, 5, 99.87, 0.05, 24288),
('2238/QPD', 'B&D cordless drill, 1/2-in.', '2008-01-20', 12, 5, 38.95, 0.05, 25595),
('23109-HB', 'Claw hammer', '2008-01-20', 23, 10, 9.95, 0.10, 21225),
('29114-AA', 'Sledge hammer, 12 lb.', '2008-01-02', 8, 5, 14.40, 0.05, NULL),
('54778-2T', 'Rat-tail file, 1/8-in. fine', '2007-12-15', 43, 20, 4.99, 0.00, 21344),

```

```
(
  ('89-WRE-Q', 'Hicut chain saw, 16 in.', '2008-02-07', 11, 5, 256.99, 0.05, 24288),
  ('PVC23DRT', 'PVC pipe, 3.5-in., 8-ft', '2008-02-20', 188, 75, 5.87, 0.00, NULL),
  ('SM-18277', '1.25-in. metal screw, 25', '2008-05-01', 172, 75, 6.99, 0.00, 21225),
  ('SW-23116', '2.5-in. wd screw, 50', '2008-02-24', 237, 100, 8.45, 0.00, 21231),
  ('WR3/TT3', 'Steel matting, 4x8x1/16, 5 mesh', '2008-01-17', 18, 5, 119.95, 0.10, 25595);
```

```
CREATE TABLE VENDOR (
  V_CODE INT PRIMARY KEY,
  V_NAME VARCHAR(100) NOT NULL,
  V_CONTACT VARCHAR(50),
  V_AREACODE CHAR(3),
  V_PHONE VARCHAR(10),
  V_STATE CHAR(2),
  V_ORDER CHAR(1) CHECK (V_ORDER IN ('Y', 'N'));
INSERT INTO VENDOR
(V_CODE, V_NAME, V_CONTACT, V_AREACODE, V_PHONE, V_STATE,
V_ORDER) VALUES(21225, 'Bryson, Inc.', 'Smithson', '615', '223-3234', 'TN',
'Y'),(21226, 'Superloo, Inc.', 'Flushing', '904', '215-8995', 'FL', 'N'),(21231, 'D&E
Supply', 'Singh', '615', '228-3245', 'TN', 'Y'),(21344, 'Gomez Bros.', 'Ortega', '615',
'889-2548', 'KY', 'N'),(22567, 'Dome Supply', 'Smith', '901', '678-1419', 'GA',
'N'),(23119, 'Randsets Ltd.', 'Anderson', '901', '678-3998', 'GA', 'Y'),(24004,
'Brackman Bros.', 'Browning', '615', '228-1410', 'TN', 'N'),(24288, 'ORDVA, Inc.',
'Hakford', '615', '889-1234', 'TN', 'Y'),(25443, 'B&K, Inc.', 'Smith', '904', '227-0093', 'FL',
'N'),(25501, 'Davis Supplies', 'Smythe', '615', '890-3529', 'TN', 'N'),(25595, 'Rubicon
Systems', 'Orton', '904', '456-0092', 'FL', 'Y');
```

4 EXERCÍCIO 16 ATÉ 33

16. Escreva uma consulta que conte o número de faturas.

```
SELECT COUNT(*) AS TotalFaturas
FROM INVOICE;
```

	TotalFaturas
1	8

17. Escreva uma consulta que conte o número de clientes com saldo superior a US\$ 500.

```
SELECT COUNT(*) AS TotalClientes
FROM CUSTOMER
WHERE CUS_BALANCE > 500;
```

	TotalClientes
1	2

18. Gere uma listagem de todas as compras feitas por clientes, utilizando como orientação o resultado apresentado na Figura P7.16. (Sugestão: Utilize a cláusula ORDER BY para ordenar as linhas resultantes apresentadas na Figura P7.16.)

```
SELECT
```

```

INVOICE.CUS_CODE,
INVOICE.INV_NUMBER,
INVOICE.INV_DATE,
PRODUCT.P_DESCRIPT,
LINE.LINE_UNITS,
LINE.LINE_PRICE
FROM INVOICE
JOIN LINE ON INVOICE.INV_NUMBER = LINE.INV_NUMBER
JOIN PRODUCT ON LINE.P_CODE = PRODUCT.P_CODE
ORDER BY
    INVOICE.CUS_CODE ASC,
    INVOICE.INV_NUMBER ASC,
    PRODUCT.P_DESCRIPT
AS

```

	CUS_CODE	INV_NUMBER	INV_DATE	P_DESCRIPT	LINE_UNITS	LINE_PRICE
1	10011	1002	2008-03-16	Rat-tail file, 1/8-in. fine	2	4.99
2	10011	1004	2008-03-17	Claw hammer	2	9.95
3	10011	1004	2008-03-17	Rat-tail file, 1/8-in. fine	3	4.99
4	10011	1008	2008-03-17	Claw hammer	1	9.95
5	10011	1008	2008-03-17	PVC pipe, 3.5-in., 8-ft	5	5.87
6	10011	1008	2008-03-17	Steel matting, 4x8x1/16, 5 mesh	3	119.95
7	10012	1003	2008-03-16	7.25-in. pwr. saw blade	5	14.99
8	10012	1003	2008-03-16	B&D cordless drill, 1/2-in.	1	38.95
9	10012	1003	2008-03-16	Hrd. cloth, 1/4-in., 2x50	1	39.95
10	10014	1001	2008-03-16	7.25-in. pwr. saw blade	1	14.99
11	10014	1001	2008-03-16	Claw hammer	1	9.95
12	10014	1006	2008-03-17	1.25-in. metal screw, 25	3	6.99
13	10014	1006	2008-03-17	B&D cordless drill, 1/2-in.	1	109.92
14	10014	1006	2008-03-17	Claw hammer	1	9.95
15	10014	1006	2008-03-17	Hicut chain saw, 16 in.	1	256.99
16	10015	1007	2008-03-17	7.25-in. pwr. saw blade	2	14.99
17	10015	1007	2008-03-17	Rat-tail file, 1/8-in. fine	1	4.99
18	10018	1005	2008-03-17	PVC pipe, 3.5-in., 8-ft	12	5.87

19. Utilizando como orientação o resultado apresentado na Figura P7.17, gere uma lista de compras de clientes, incluindo os subtotais de cada número de linha de fatura. (Sugestão: Modifique o formato da consulta utilizada para produzir a lista de compras de clientes do Problema 18, exclua a coluna INV_DATE e adicione o atributo derivado (computado) LINE_UNITS * LINE_PRICE para calcular os subtotais.)

```

SELECT
    INVOICE.CUS_CODE,
    INVOICE.INV_NUMBER,
    PRODUCT.P_DESCRIPT,
    LINE.LINE_UNITS,
    LINE.LINE_PRICE,
    (LINE.LINE_UNITS * LINE.LINE_PRICE) AS Subtotal
FROM INVOICE
JOIN LINE ON INVOICE.INV_NUMBER = LINE.INV_NUMBER
JOIN PRODUCT ON LINE.P_CODE = PRODUCT.P_CODE

```

ORDER BY

INVOICE.CUS_CODE ASC,
 INVOICE.INV_NUMBER ASC,
 PRODUCT.P_DESCRIPTION ASC;

	CUS_CODE	INV_NUMBER	P_DESCRIPTION	LINE_UNITS	LINE_PRICE	Subtotal
1	10011	1002	Rat-tail file, 1/8-in. fine	2	4.99	9.98
2	10011	1004	Claw hammer	2	9.95	19.90
3	10011	1004	Rat-tail file, 1/8-in. fine	3	4.99	14.97
4	10011	1008	Claw hammer	1	9.95	9.95
5	10011	1008	PVC pipe, 3.5-in., 8-ft	5	5.87	29.35
6	10011	1008	Steel matting, 4x8x1/16, 5 mesh	3	119.95	359.85
7	10012	1003	7.25-in. pwr. saw blade	5	14.99	74.95
8	10012	1003	B&D cordless drill, 1/2-in.	1	38.95	38.95
9	10012	1003	Hrd. cloth, 1/4-in., 2x50	1	39.95	39.95
10	10014	1001	7.25-in. pwr. saw blade	1	14.99	14.99
11	10014	1001	Claw hammer	1	9.95	9.95
12	10014	1006	1.25-in. metal screw, 25	3	6.99	20.97
13	10014	1006	B&D cordless drill, 1/2-in.	1	109.92	109.92
14	10014	1006	Claw hammer	1	9.95	9.95
15	10014	1006	Hicut chain saw, 16 in.	1	256.99	256.99
16	10015	1007	7.25-in. pwr. saw blade	2	14.99	29.98
17	10015	1007	Rat-tail file, 1/8-in. fine	1	4.99	4.99
18	10018	1005	PVC pipe, 3.5-in., 8-ft	12	5.87	70.44

20. Modifique a consulta utilizada no Problema 19 para produzir o resumo apresentado na Figura P7.18.

SELECT

CUSTOMER.CUS_CODE,
 CUSTOMER.CUS_BALANCE,
 SUM(LINE.LINE_UNITS * LINE.LINE_PRICE) AS [Total Purchases]

FROM CUSTOMER

JOIN INVOICE ON CUSTOMER.CUS_CODE = INVOICE.CUS_CODE

JOIN LINE ON INVOICE.INV_NUMBER = LINE.INV_NUMBER

GROUP BY

CUSTOMER.CUS_CODE,
 CUSTOMER.CUS_BALANCE

ORDER BY

CUSTOMER.CUS_CODE;

	CUS_CODE	CUS_BALANCE	Total Purchases
1	10011	0.00	444.00
2	10012	345.86	153.85
3	10014	0.00	422.77
4	10015	0.00	34.97
5	10018	216.55	70.44

21. Modifique a consulta do Problema 20 de modo a incluir o número de compras individuais de produtos feitas por cliente. (Em outras palavras, se a fatura do cliente basear-se em três produtos, um por número de linha (LINE_NUMBER), deve-se contar três compras de produtos. Observe que, nos dados da fatura original, o

cliente 10011 gerou três faturas que continham um total de seis linhas, cada uma representando a compra de um produto.) Os valores de seus resultados devem coincidir com os apresentados na Figura P7.19.

```
SELECT
    CUSTOMER.CUS_CODE,
    CUSTOMER.CUS_BALANCE,
    SUM(LINE.LINE_UNITS * LINE.LINE_PRICE) AS [Total Purchases],
    COUNT(*) AS [Number of Purchases]
FROM CUSTOMER
JOIN INVOICE ON CUSTOMER.CUS_CODE = INVOICE.CUS_CODE
JOIN LINE ON INVOICE.INV_NUMBER = LINE.INV_NUMBER
GROUP BY
    CUSTOMER.CUS_CODE,
    CUSTOMER.CUS_BALANCE
ORDER BY
    CUSTOMER.CUS_CODE;
```

	CUS_CODE	CUS_BALANCE	Total Purchases	Number of Purchases
1	10011	0.00	444.00	6
2	10012	345.86	153.85	3
3	10014	0.00	422.77	6
4	10015	0.00	34.97	2
5	10018	216.55	70.44	1

22. Utilize uma consulta para computar a quantia média de compras por produto feita por cliente. (*Sugestão:* Utilize os resultados do Problema 21 como base desta consulta.) Os valores de seus resultados devem coincidir com os apresentados na Figura P7.20. Observe que a quantia média de compras é igual ao total de compras dividido pelo número de compras.

```
SELECT
    CUSTOMER.CUS_CODE,
    CUSTOMER.CUS_BALANCE,
    SUM(LINE.LINE_UNITS * LINE.LINE_PRICE) AS [Total Purchases],
    COUNT(*) AS [Number of Purchases],
    (SUM(LINE.LINE_UNITS * LINE.LINE_PRICE) / COUNT(*)) AS [Average
Purchase Amount]
FROM CUSTOMER
JOIN INVOICE ON CUSTOMER.CUS_CODE = INVOICE.CUS_CODE
JOIN LINE ON INVOICE.INV_NUMBER = LINE.INV_NUMBER
GROUP BY
    CUSTOMER.CUS_CODE,
    CUSTOMER.CUS_BALANCE
ORDER BY
    CUSTOMER.CUS_CODE;
```

	CUS_CODE	CUS_BALANCE	Total Purchases	Number of Purchases	Average Purchase Amount
1	10011	0.00	444.00	6	74.000000
2	10012	345.86	153.85	3	51.283333
3	10014	0.00	422.77	6	70.461666
4	10015	0.00	34.97	2	17.485000
5	10018	216.55	70.44	1	70.440000

23. Crie uma consulta para produzir as compras totais por fatura, gerando os resultados apresentados na Figura P7.21. O total da fatura é a soma das compras de produtos em LINE que correspondem à fatura (INVOICE).

```
SELECT
    INVOICE.INV_NUMBER,
    SUM(LINE.LINE_UNITS * LINE.LINE_PRICE) AS [Invoice Total]
FROM INVOICE
JOIN LINE ON INVOICE.INV_NUMBER = LINE.INV_NUMBER
GROUP BY
    INVOICE.INV_NUMBER
ORDER BY
    INVOICE.INV_NUMBER;
```

	INV_NUMBER	Invoice Total
1	1001	24.94
2	1002	9.98
3	1003	153.85
4	1004	34.87
5	1005	70.44
6	1006	397.83
7	1007	34.97
8	1008	399.15

24. Utilize uma consulta para mostrar as faturas e totais de faturas conforme apresentado na Figura P7.22. (*Sugestão: Agrupe por CUS_CODE.*)

```
SELECT
    INVOICE.CUS_CODE,
    INVOICE.INV_NUMBER,
    SUM(LINE.LINE_UNITS * LINE.LINE_PRICE) AS [Invoice Total]
FROM INVOICE
JOIN LINE ON INVOICE.INV_NUMBER = LINE.INV_NUMBER
GROUP BY
    INVOICE.CUS_CODE,
    INVOICE.INV_NUMBER
ORDER BY
    INVOICE.CUS_CODE,
    INVOICE.INV_NUMBER;
```

	CUS_CODE	INV_NUMBER	Invoice Total
1	10011	1002	9.98
2	10011	1004	34.87
3	10011	1008	399.15
4	10012	1003	153.85
5	10014	1001	24.94
6	10014	1006	397.83
7	10015	1007	34.97
8	10018	1005	70.44

25. Escreva uma consulta que produza o número de faturas e as quantias totais de compras por cliente, utilizando como orientação o resultado da Figura P7.23. (Compare esse resumo aos resultados apresentados no Problema 24.)

```
SELECT
    INVOICE.CUS_CODE,
    COUNT(DISTINCT INVOICE.INV_NUMBER) AS [Number of Invoices],
    SUM(LINE.LINE_UNITS * LINE.LINE_PRICE) AS [Total Customer Purchases]
FROM INVOICE
JOIN LINE ON INVOICE.INV_NUMBER = LINE.INV_NUMBER
GROUP BY
    INVOICE.CUS_CODE
ORDER BY
    INVOICE.CUS_CODE;
```

	CUS_CODE	Number of Invoices	Total Customer Purchases
1	10011	3	444.00
2	10012	1	153.85
3	10014	2	422.77
4	10015	1	34.97
5	10018	1	70.44

26. Utilizando os resultados do Problema 25 como base, escreva uma consulta que gere o número total de faturas, o total de todas as faturas, o menor e maior valor de fatura, e o valor médio de todas as faturas. (*Sugestão:* Verifique o resultado da figura do Problema 25.) Seu resultado deve corresponder à Figura P7.24.

```
SELECT
    SUM([Number of Invoices]) AS [Total Invoices],
    SUM([Total Customer Purchases]) AS [Total Sales],
    MIN([Total Customer Purchases]) AS [Minimum Sale],
    MAX([Total Customer Purchases]) AS [Largest Sale],
    CAST(AVG([Total Customer Purchases]) AS DECIMAL(10,2)) AS [Average Sale]
FROM (
    SELECT
        INVOICE.CUS_CODE,
        COUNT(DISTINCT INVOICE.INV_NUMBER) AS [Number of Invoices],
        SUM(LINE.LINE_UNITS * LINE.LINE_PRICE) AS [Total Customer Purchases]
    FROM INVOICE
    JOIN LINE ON INVOICE.INV_NUMBER = LINE.INV_NUMBER
    GROUP BY
        INVOICE.CUS_CODE
) AS ResumoClientes;
```

	Total Invoices	Total Sales	Minimum Sale	Largest Sale	Average Sale
1	8	1126.03	34.97	444.00	225.21

27. Liste as características dos saldos dos clientes que fizeram compras durante o ciclo atual de faturas - ou seja, dos clientes que aparecem na tabela INVOICE. Os resultados dessa consulta são apresentados na Figura P7.25.

```
SELECT
    CUS_CODE,
    CUS_BALANCE
```

```

FROM CUSTOMER
WHERE CUS_CODE IN (
    SELECT CUS_CODE
    FROM INVOICE
);

```

	CUS_CODE	CUS_BALANCE
1	10011	0.00
2	10012	345.86
3	10014	0.00
4	10015	0.00
5	10018	216.55

28. Utilizando os resultados da consulta criada no Problema 27, forneça um resumo das características dos saldos de clientes, conforme a Figura P7.26.

```

SELECT
    MIN(CUS_BALANCE) AS [Minimum Balance],
    MAX(CUS_BALANCE) AS [Maximum Balance],
    CAST(AVG(CUS_BALANCE) AS DECIMAL(10,2)) AS [Average Balance]
FROM CUSTOMER
WHERE CUS_CODE IN (
    SELECT CUS_CODE
    FROM INVOICE
);

```

	Minimum Balance	Maximum Balance	Average Balance
1	0.00	345.86	112.48

29. Crie uma consulta que encontre as características de saldos de todos os clientes, incluindo o total de saldos a receber. Os resultados dessa consulta são apresentados na Figura P7.27.

```

SELECT
    SUM(CUS_BALANCE) AS [Total Balance],
    MIN(CUS_BALANCE) AS [Minimum Balance],
    MAX(CUS_BALANCE) AS [Maximum Balance],
    CAST(AVG(CUS_BALANCE) AS DECIMAL(10,2)) AS [Average Balance]
FROM CUSTOMER;

```

	Total Balance	Minimum Balance	Maximum Balance	Average Balance
1	2089.28	0.00	768.93	208.93

30. Obtenha a listagem de clientes que não fizeram compras durante o período de faturamento. Seus resultados devem coincidir com os apresentados na Figura P7.28.

```

SELECT
    CUS_CODE,
    CUS_BALANCE
FROM CUSTOMER
WHERE CUS_CODE NOT IN (
    SELECT CUS_CODE
    FROM INVOICE
);

```

	CUS_CODE	CUS_BALANCE
1	10010	0.00
2	10013	536.75
3	10016	221.19
4	10017	768.93
5	10019	0.00

31. Obtenha o resumo dos saldos de todos os clientes que não fizeram compras durante o período atual de faturamento. Os resultados são apresentados na Figura P7.29.

```
SELECT
    SUM(CUS_BALANCE) AS [Total Balance],
    MIN(CUS_BALANCE) AS [Minimum Balance],
    MAX(CUS_BALANCE) AS [Maximum Balance],
    CAST(AVG(CUS_BALANCE) AS DECIMAL(10,2)) AS [Average Balance]
FROM CUSTOMER
WHERE CUS_CODE NOT IN (
    SELECT CUS_CODE
    FROM INVOICE
);
```

	Total Balance	Minimum Balance	Maximum Balance	Average Balance
1	1526.87	0.00	768.93	305.37

32. Crie uma consulta para produzir o resumo do valor dos produtos atualmente em estoque. Observe que o valor de cada produto é obtido pela multiplicação das unidades atualmente em estoque pelo preço unitário. Utilize a cláusula ORDER BY para obter a ordem apresentada na Figura P7.30.

```
SELECT

    PRODUCT.P_DESCRIPT,

    PRODUCT.P_QOH,

    PRODUCT.P_PRICE,

    (PRODUCT.P_QOH * PRODUCT.P_PRICE) AS Subtotal

FROM PRODUCT

ORDER BY

    PRODUCT.P_CODE;
```

	P_DESCRIPT	P_QOH	P_PRICE	Subtotal
1	Power painter, 15 psi, 3-nozzle	8	109.99	879.92
2	7.25-in. pwr. saw blade	32	14.99	479.68
3	8.00-in. pwr. saw blade	18	17.49	314.82
4	Hrd. cloth, 1/4-in., 2x50	15	39.95	599.25
5	Hrd. cloth, 1/8-in., 3x50	23	43.99	1011.77
6	B&D jigsaw, 12-in. blade	8	109.92	879.36
7	B&D jigsaw, 8-in. blade	6	99.87	599.22
8	B&D cordless drill, 1/2-in.	12	38.95	467.40
9	Claw hammer	23	9.95	228.85
10	Sledge hammer, 12 lb.	8	14.40	115.20
11	Rat-tail file, 1/8-in. fine	43	4.99	214.57
12	Hicut chain saw, 16 in.	11	256.99	2826.89
13	PVC pipe, 3.5-in., 8-ft	188	5.87	1103.56
14	1.25-in. metal screw, 25	172	6.99	1202.28
15	2.5-in. wd screw, 50	237	8.45	2002.65
16	Steel matting, 4x8x1/16, 5 mesh	18	119.95	2159.10

33. Utilizando os resultados da consulta criada no Problema 32, obtenha o valor total do estoque de produtos. Os resultados são apresentados na Figura P7.31.

SELECT

SUM(PRODUCT.P_QOH * PRODUCT.P_PRICE) AS [Total Value of Inventory]

FROM PRODUCT;

	Total Value of Inventory
1	15084.52

5 CONCLUSÃO

A realização desta segunda lista de exercícios cumpriu de forma satisfatória os objetivos práticos e teóricos propostos para a disciplina de Banco de Dados 2. Através da estruturação do estudo de caso e da resolução das questões 16 a 33, foi possível aprofundar e consolidar conceitos avançados de manipulação de dados via SQL, com destaque para a aplicação de múltiplas junções (*JOINS*), subconsultas complexas, funções de agregação e agrupamentos lógicos. A execução dos scripts no Sistema Gerenciador de Banco de Dados (SGBD) evidenciou a importância de construir consultas precisas para a extração de métricas gerenciais e relatórios financeiros. Conclui-se, portanto, que as técnicas aplicadas nesta etapa reforçam o raciocínio lógico necessário para a administração de bancos de dados relacionais, formando uma base técnica sólida para o avanço da disciplina no decorrer do semestre.

